Harran Üniversitesi · Mühendislik Fakültesi
Bilgisayar Mühendisliği ve Yazılım Mühendisliği Lisans Programları
YAZILIM GEÇERLEME VE SINAMA · 2026–2027 Güz Yarıyılı
Dersin Öğretim Üyesi: Dr. Öğr. Üyesi Harun ÇİĞ

Hafta 1 — Yazılım Testinin Temelleri

Test neyi gösterir, neyi asla gösteremez?

Bu haftanın hedefleri

Bu haftanın sonunda:

- Bir olayda **insan hatasını**, **hatayı** ve **arızayı** birbirinden ayırabilirsiniz.
- Testin neyi gösterip neyi asla gösteremeyeceğini bir örnekle savunabilirsiniz.
- **Test etme** ile **hata ayıklamayı**, **doğrulama** ile **geçerlemeyi** ayırt edebilirsiniz.
- Yedi test ilkesini BiletKapısı'ndan bir durumla eşleştirebilirsiniz.

> **Terimler hakkında:** Bu derste ISTQB'nin resmî Türkçe müfredatındaki (Turkish Testing Board çevirisi) terimleri kullanıyoruz. Her terimin İngilizcesi ilk geçtiği yerde parantez içinde verilir; iş görüşmeleri çoğu zaman İngilizce terimlerle yapılır.

1. Neden test ediyoruz?

Haftanın arızası: CrowdStrike, 19 Temmuz 2024

CrowdStrike Falcon, Windows bilgisayarlarda işletim sisteminin çekirdeğinde çalışan bir güvenlik yazılımıdır. 19 Temmuz 2024'te (04:09 UTC) şirket, yeni saldırı kalıplarını tanımak için bir **içerik güncellemesi** gönderdi: kod değil, bir yapılandırma veri dosyası (Channel File 291). Güncelleme 78 dakika sonra geri alındı, ama çöken bilgisayarlar kendiliğinden düzelmedi. Microsoft'un tahminine göre yaklaşık **8,5 milyon** Windows cihazı etkilendi.

Şirketin kök neden raporuna göre üç sorun birlikte oluştu: yeni şablon 21 girdi alanı tanımlıyordu ama yorumlayıcıyı çağıran kod 20 değer veriyordu; yorumlayıcıda dizi sınırı denetimi yoktu; testlerde ise 21. alan hep "joker" (her şeyle eşleşen) değerle kullanılmıştı. Hata Şubat 2024'ten beri sahadaydı ve önceki dağıtımlar sorunsuz geçmişti. 19 Temmuz'da 21. alanı ilk kez joker olmayan bir kural kullandı; yorumlayıcı dizinin sınırı dışından okudu ve Windows çöktü.

> **Bu haftanın sorusu:** Hata beş ay boyunca sahadaydı ve testlerin hepsi geçiyordu. Test bu hatayı neden göremedi? (İpucu: bölüm 2.2'deki dört koşul ve bölüm 3'teki ilke 1 ile ilke 5.)

Dört arıza, dört ders

Olay	Ne oldu?	Test açısından ders
Therac-25 (1985–87)	Radyoterapi cihazında 6 kazada aşırı radyasyon; 3 ölüm aşırı dozla ilişkili. Yarış koşulu ve 1 baytlık sayaç taşması.	Donanım güvenlik kilitleri yazılıma güvenilerek kaldırılmıştı; eşzamanlılık hataları nadiren ve öngörülemez biçimde görünür.
Ariane 501 (1996)	İlk uçuşta kalkıştan ~30 s sonra parçalandı. Ariane 4'ten alınan kodda 64 bit kayan noktalı sayıdan 16 bit tamsayıya dönüşüm taşması.	Yeniden kullanılan kod, yeni bağlamın (Ariane 5 yörüngesi) verisiyle test edilmemişti.
Knight Capital (2012)	Yeni kod 8 sunucudan birine kopyalanmadı; eski bir bayrak yıllardır kullanılmayan kodu uyandırdı. 45 dakikada 460 milyon $'dan fazla zarar.	Eski kod yeniden test edilmemişti; dağıtım da test edilmesi gereken bir süreçtir.
CrowdStrike (2024)	Bir veri dosyası 21 alan beklerken kod 20 değer verdi; ~8,5 milyon Windows cihazı çöktü.	Testler hep aynı veriyle koşulmuştu; güncelleme kademeli değil, herkese aynı anda gönderilmişti.

Kalitesizliğin bedeli

Sık alıntılanan iki kestirim var. NIST için hazırlanan 2002 raporu, ABD'de **yetersiz test altyapısının** yıllık maliyetini 59,5 milyar $ olarak hesaplar; bunun ~22,2 milyar $'ı altyapı iyileştirmeleriyle azaltılabilir. CISQ'nun 2022 raporu ise ABD'de kötü yazılım kalitesinin maliyetini en az 2,41 trilyon $ olarak verir. İkisi de ölçüm değil kestirimdir; kaynağını okumadan "yazılım hataları yılda X dolara mal oluyor" demeyin.

Yapay zekâ kod üretimini ucuzlattı ama doğrulamayı ucuzlatmadı. Stack Overflow 2025 geliştirici anketinde YZ araçlarını kullananların %66'sı en büyük sorunu "neredeyse doğru ama tam değil" çözümler olarak gösterdi; YZ çıktısının doğruluğuna güvenenlerin oranı bir yılda %43'ten %33'e düştü. PractiTest'in 2026 raporuna göre test uzmanları mesleğin bir numaralı becerisini **eleştirel düşünme** olarak görüyor.

> **"Geç bulunan hata 100 kat pahalıdır" mı?** Erken test genellikle daha ucuzdur (ISTQB ilke 3). Ama "100 kat" sabit bir çarpan değildir. Boehm ve Basili (2001) bu oranı 1970'lerin büyük sistemleri için "çoğu zaman" geçerli sayar ve küçük sistemlerde ~5:1 olduğunu söyler; Menzies ve arkadaşları (2017) 171 modern projede böyle bir etki bulamadı. Bir rakam duyduğunuzda önce kaynağını sorun.

2. Temel kavramlar

2.1 Test etme nedir?

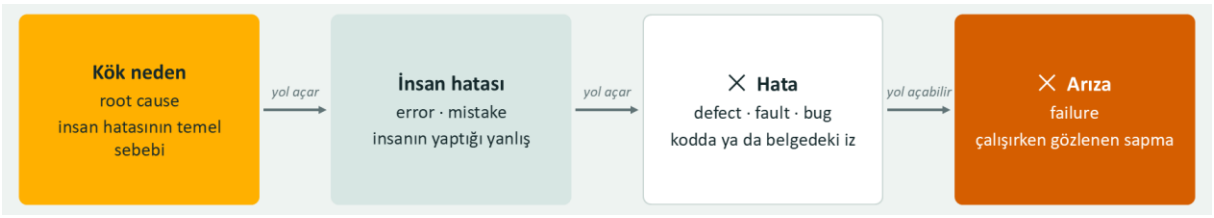
Test etme (testing), yazılımdaki hataları bulmak ve kalitesini değerlendirmek için yürütülen bir dizi etkinliktir. Yalnızca programı çalıştırmak değildir:

- **Statik test** programı çalıştırmadan yapılır: gereksinim, tasarım ve kod incelemesi, statik analiz. Hatayı doğrudan bulur (4. hafta).
- **Dinamik test** programı çalıştırarak yapılır: girdi verilir, çıktı gözlenir ve beklenen sonuçla karşılaştırılır. Arızayı gösterir; hatanın yerini bulmak ayrı bir iştir (5–12. haftalar).

Test edilen şeye **test nesnesi** (test object), testlerin türetildiği belgeye **test esası** (test basis) denir. BiletKapısı'nda iptal kuralı bir test nesnesi, spesifikasyondaki G-22 gereksinimi ise onun test esasıdır.

2.2 İnsan hatası → hata → arıza

İnsanlar **insan hatası** (error, mistake) yapar. Bu, üründe — kodda ya da bir belgede — bir **hataya** (defect; eş anlamlıları fault, bug) yol açar. Hatalı kod çalıştırıldığında bir **arızaya** (failure) yol açabilir: sistemin beklenen davranıştan gözlenebilir biçimde sapmasına. İnsan hatasına yol açan temel sebebe **kök neden** (root cause) denir.



Şekil 1. Kök neden, insan hatası, hata ve arıza (ISTQB CTFL v4.0.1 §1.2.3'e göre çizildi)

BiletKapısı örneği. Gereksinim G-22: "ONAYLANDI durumundaki rezervasyon, etkinlik başlangıcına **en az** 24 saat varken iptal edilebilir." Geliştirici bunu "24 saatten **fazla**" diye okur (insan hatası) ve kodda `kalanDakika > 1440` yazar (hata). Etkinliğe tam 24 saat kala iptal etmek isteyen kullanıcı reddedilir (arıza). Kök neden: gereksinim incelemesinde "tam 1440 dakika kala ne olur?" diye bir sınır örneği sorulmamış olmasıdır.

Her hata arızaya dönüşmez. Bir hatanın arıza olarak görülmesi için dört koşul birlikte sağlanmalıdır (Ammann ve Offutt'un modeli):



Şekil 2. Bir hatanın arıza olarak görülmesinin dört koşulu (Ammann & Offutt, Introduction to Software Testing, 2. bs. — Türkçeleştirilerek yeniden çizildi)

BiletKapısı'ndaki hata, 1441 ya da 1439 dakikayla yapılan testlerde görünmez: > ve >= bu değerlerde aynı sonucu verir. Hatayı yalnız **tam 1440 dakika** açığa çıkarır. Hataların sınırlarda saklanması, 6. haftada göreceğimiz sınır değer analizinin dayanağıdır. CrowdStrike'ta da hata, 21. alanı okuyan bir veri gelene kadar ulaşılabilir değildi.

Her arızanın nedeni bir hata değildir. Çevresel koşullar da arızaya yol açabilir: radyasyon, elektromanyetik alan, donanım arızası, dolu disk, kesilen ağ. 1947'de Harvard Mark II'nin bir rölesine sıkışan güve buna bir örnektir: arıza vardı, ama bir insan hatası yoktu.

2.3 Terimler: resmî Türkçe karşılıklar

İngilizce	Resmî Türkçe (ISTQB TR)	Sık karıştırılan	BiletKapısı örneği
error, mistake	insan hatası	gündelik dildeki "hata"	"En az 24 saat"i "fazla" diye okumak
defect, fault, bug	hata	"arıza", "kusur"	Kodda > yazılmış olması
failure	arıza	"başarısızlık", "bug"	Tam 24 saatte iptalin reddedilmesi
root cause	kök neden	"suçlu"	İncelemede sınır örneğinin sorulmaması
testing / debugging	test etme / hata ayıklama	ikisini aynı sanmak	Arızayı göstermek / hatayı bulup gidermek
verification / validation	doğrulama / sağlama	dersin adındaki "geçerleme" = sağlama	Belirtime uygun mu / kullanıcıya uygun mu

> **Dikkat:** Resmî Türkçe müfredatın 1.2.3 bölümündeki bir cümlede *fault* "arıza", *failure* "başarısızlık" diye çevrilmiştir; bu, müfredatın kendi anahtar kelime listesiyle çelişir. Sınavda ve bu derste anahtar kelime eşlemesi esastır: **defect = hata**, **failure = arıza**.

2.4 Test etme ve hata ayıklama

Test etme ve hata ayıklama ayrı etkinliklerdir. **Test etme** bir arızayı tetikler (dinamik test) ya da hatayı doğrudan bulur (statik test). **Hata ayıklama** (debugging) arızanın nedenini bulup giderir; üç adımdan oluşur: arızanın yeniden oluşturulması, teşhis (hatanın bulunması) ve düzeltme. Düzeltmeden sonra yine test gelir:

- **Onaylama testi** (confirmation testing): asıl hata giderildi mi? Tercihen ilk testi yapan kişi yapar.
- **Regresyon testi** (regression testing): düzeltme başka bir yeri bozdu mu? Knight Capital'de eski kod yeniden test edilmemişti.

2.5 Doğrulama ve geçerleme

- **Doğrulama** (verification): "Ürünü **doğru mu** yapıyoruz?" Ölçüt, belirtilen gereksinimlerdir. Örnek: tam 1440 dakika kala iptal kabul ediliyor mu?
- **Geçerleme** (validation; ISTQB Türkçesinde **sağlama**): "**Doğru ürünü** mü yapıyoruz?" Ölçüt, kullanıcı ve paydaş ihtiyaçlarıdır. Örnek: 24 saat kuralı kullanıcının gerçekten işine yarıyor mu?

Bütün testleri geçen, belirtime tıpatıp uyan bir ürün yine de kullanıcının işine yaramayabilir. Test etme ikisini de içerir.

2.6 Test hedefleri

ISTQB dokuz genel test hedefi sayar. Üç grupta toplanabilirler:

- **Bulmak:** iş ürünlerini (gereksinim, tasarım, kod) değerlendirmek; arızaları tetiklemek ve hataları bulmak; gerekli kapsamı sağlamak.
- **Doğrulamak ve sağlamak:** belirtilen gereksinimleri doğrulamak; yasal, sözleşmesel ve düzenleyici gereksinimlere uyumu doğrulamak; sistemin paydaşların beklediği gibi çalıştığının sağlamasını yapmak.
- **Bilgi ve güven üretmek:** kalite kriterlerini karşılayamama riskini azaltmak; paydaşlara karar için yeterli bilgi sunmak; kaliteye güven oluşturmak.

Hedefler bağlama göre değişir: bileşen testinde hata bulmak, kabul testinde güven oluşturmak ağır basar.

2.7 Test etme ve kalite güvence

Test etme ürün odaklı ve düzelticidir; kalite kontrolünün bir biçimidir. **Kalite güvence** (KG) süreç odaklı ve önleyicidir: iyi bir süreç doğru izlenirse iyi bir ürün çıkacağını varsayar. Örnek: G-22 için sınır testi yazmak test etmedir; her gereksinim incelemesinde sınır örneği sormayı kural hâline getirmek kalite güvencedir.

3. Yedi test ilkesi

#	Resmî Türkçe ad (ISTQB TR)	İngilizce	Tek cümleyle
1	Testin amacı, yazılımda hataların olduğunu göstermektir; yazılımda hata kalmadığını ispatlamak değildir	Testing shows the presence, not the absence of defects	Test hata bulabilir, yokluğunu kanıtlayamaz
2	Yazılımı %100 test etmek imkansızdır	Exhaustive testing is impossible	Her girdiyi denemek mümkün değil; teknik ve öncelik gerekir
3	Erken test, zaman ve para tasarrufu sağlar	Early testing saves time and money	Erken giderilen hata yeni hata doğurmaz
4	Hatalar yazılımın belli alanlarında yoğunlaşır	Defects cluster together	Hatalar az sayıda bileşende toplanır
5	Antibiyotik direnci	Tests wear out	Aynı testler zamanla yeni hata bulamaz
6	Yazılım testi, projenin bağlamına, koşullarına göre değişiklik gösterir	Testing is context dependent	Tek bir doğru yaklaşım yok
7	Yeni hata bulamıyoruz başarılı bir yazılım elde ettik yanılgısı	Absence-of-defects fallacy	Hata bulmamak, doğru ürünü yaptık demek değil

İlke 2'ye bir örnek. BiletKapısı'nın 200 koltuklu salonunda bir işlemde 1–6 koltuk seçilebilir (G-11). Sıra önemsiz olduğundan seçim sayısı C(200,1) + … + C(200,6) = **85.010.294.790**'dır. Her deneme 1 ms sürse, tek bir etkinlik için yaklaşık **2,7 yıl** gerekir — kampanya kodu, öğrenci indirimi ve zaman kuralları hesaba katılmadan.

İlkeler olaylarda. CrowdStrike'ta testler beş ay boyunca geçti (ilke 1) ve 12 senaryoluk sabit bir küme hep aynı şeyi sınadı (ilke 5). Ariane 4'te doğru çalışan kod Ariane 5'te çöktü (ilke 6). ISTQB v4.0'da iki ilkenin adı değişti: "pesticide paradox" yerine "tests wear out", "absence-of-errors fallacy" yerine "absence-of-defects fallacy"; eski kaynaklarda eski adları görebilirsiniz.

4. Derste: şikâyetten teste

Derste G-22 şikâyetini bir JUnit testine çevirdik. Test, kullanıcının gördüğü arızayı tekrarlanabilir bir deneye dönüştürür; beklenen sonuç koddan değil gereksinimden gelir:

```
@Test
@DisplayName("Tam 24 saat kala onaylı rezervasyon iptal edilebilmeli (G-22)")
void tamYirmiDortSaatKalaIptalKabulEdilmeli() {
    LocalDateTime etkinlik = LocalDateTime.of(2026, 10, 10, 20, 0);
    LocalDateTime simdi    = LocalDateTime.of(2026, 10,  9, 20, 0);  // tam 1440 dk önce
    boolean sonuc = IptalKurali.iptalEdilebilirMi(
            RezervasyonDurumu.ONAYLANDI, simdi, etkinlik);
    assertTrue(sonuc, "G-22: tam 1440 dakika kala iptal kabul edilmeliydi");
}
```

Test kırmızı yandı (`expected: <true> but was: <false>`): arıza yeniden üretildi. Kodu okuyarak hatayı bulduk (teşhis), > işaretini >= yaptık (düzeltme), aynı test yeşil oldu (onaylama testi) ve bütün testler yeniden geçti (regresyon testi). Testin yapamadığı şey ise G-22'nin kullanıcı için doğru kural olup olmadığını söylemekti; o, geçerlemenin işidir.

5. Kendinizi sınayın

1. BiletKapısı'nda arama kutusuna "istanbul" yazan kullanıcı "İSTANBUL SENFONİ" konserini bulamıyor. Arızayı, olası hatayı ve olası insan hatasını yazın.
2. Bir hata kodda bulunduğu hâlde hiçbir testte arızaya yol açmayabilir. Bu nasıl mümkün olur? Dört koşuldan hareketle açıklayın.
3. "Kodu printf ekleyerek test ettim" cümlesindeki terim yanlışlığını düzeltin.
4. Aynı koltuğun iki kişiye satıldığı bir şikâyette arızanın nedeni bir yazılım hatası mı, çevresel bir koşul mu olabilir? İkisi için de birer senaryo yazın.
5. "Hiç arıza bulamadık, sürüme hazırız" diyen bir ekip lideri hangi iki ilkeyi gözden kaçırıyor olabilir?

> **Mülakat köşesi:** *"Error, defect ve failure arasındaki fark nedir?"* İyi bir cevap üç katmanlıdır: (1) üç terimin tanımı, (2) bir örnek (kendi projenizden ya da Ariane 5, CrowdStrike gibi bir olaydan), (3) incelik: her hata arızaya dönüşmez ve her arızanın nedeni bir hata değildir. Mülakatlar çoğu zaman İngilizce terimlerle yapılır; iki dilde de cevap verebilmelisiniz.

6. Dünyanın önde gelen üniversitelerinde bu konu

Dört saygın kaynak, bu haftanın iki sorusuna (test ne gösterir, hata nasıl arızaya dönüşür?) bizim notları tamamlayan birer fikirle yaklaşır.

MIT 6.102 Software Construction — "Testing" okuması. MIT'nin yazılım kurgusu dersi testi, kod incelemesi ve biçimsel akıl yürütmeyle birlikte geçerleme araçlarından biri sayar. En öğretici savı yazılımın *süreksiz* davranmasıdır: belli bir yükle sınanan köprünün daha hafif yüklere de dayanacağı varsayılabilir; yazılımda ise bir girdiden komşusuna geçerken davranış birden değişebilir; "birkaç tipik değer" güven vermez. Çare, 6. haftada göreceğimiz girdi uzayını **bölmeleme** ve sınırları seçmedir. Okuma ayrıca önce-test programlamayı (belirtim → testler → kod) ve iyi bir test kümesinin üç niteliğini (doğru, kapsamlı, küçük) tanımlar.

George Mason — Ammann ve Offutt. Şekil 2'deki dört koşul bu kitaptan geldi. İlk bölümdeki bir başka fikir, Beizer'den uyarlanan **test olgunluk düzeyleri**dir: 0. düzeyde test ile hata ayıklama ayrılmaz; 1. düzeyde amaç yazılımın çalıştığını göstermektir; 2. düzeyde çalışmadığını göstermek; 3. düzeyde riski azaltmak; 4. düzeyde test bütün ekibin kalite disiplinidir. "printf ile test ettim" cümlesi 0. düzeydir; ISTQB'nin ilke 1'i 2. düzeye, ilke 7 ise 3. düzeye karşılık gelir.

Saarland Üniversitesi / CISPA — Zeller. Andreas Zeller'in *The Debugging Book* ve ortak yazarlarla yazdığı *The Fuzzing Book* adlı açık kitapları (CC BY-NC-SA 4.0) aynı kavramları çalıştırılabilir Python defterleriyle anlatır. Zeller'in **hata → enfeksiyon → arıza** zinciri Şekil 2'nin aynısıdır: hatalı satır çalışır, program durumu "enfekte" olur, enfeksiyon çıktıya ulaşırsa arıza görünür. *The Debugging Book* hata ayıklamayı bilimsel yöntem olarak

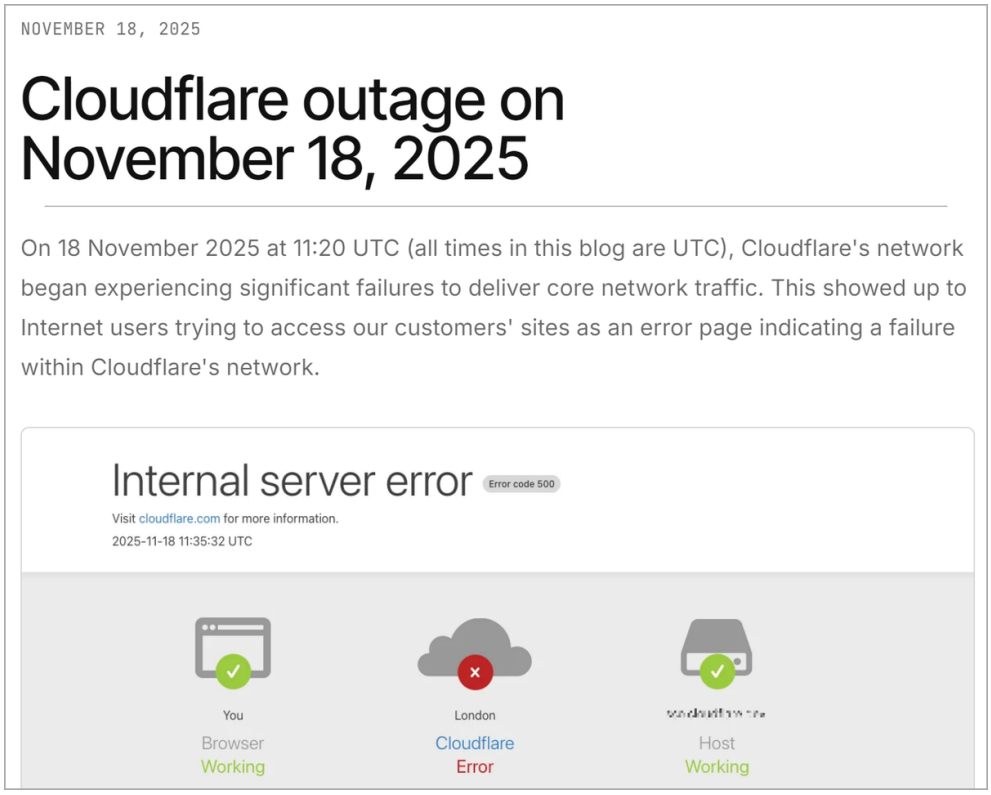
kurar: arızayı yeniden üret, hipotez kur, tahmin yaz, deney yap, gözlemle, sonuca var. *The Fuzzing Book* ise bir karekök fonksiyonunu önce elle, sonra rastgele girdilerle sınayarak tek testin neden yetmediğini gösterir.

Google — Software Engineering at Google (2020), Bölüm 11. Google'ın web sunucusu (GWS) ekibinde 2005'te üretime giden değişikliklerin %80'den fazlası kullanıcıyı etkileyen hata taşıyor ve geri alınıyordu; çözüm, her değişikliğin testiyle gelmesini zorunlu kılmak ve testleri sürekli koşmak oldu. Bölümün "Beyoncé kuralı" (beğendiysen üzerine bir test koymalıydın) ilke 1'in kurumsal hâlidir: güvendiğiniz davranışın testi yoksa biri onu bozduğunda haberiniz olmaz. Kararsız (flaky) testlerin oranı %1'e yaklaştığında testler değerini yitirmeye başlar.

Kavram	ISTQB (TTB)	Zeller	Ammann ve Offutt
Koddaki yanlış	hata (defect)	defect	fault
Yanlışlaşan program durumu	—	infection	infection (2. koşul)
Gözlenen sapma	arıza (failure)	failure	failure

Güncel (2024): CrowdStrike'ın Ağustos 2024 kök neden raporundaki önlemler bugünün sektör pratiğini özetler: derleme zamanında alan sayısı doğrulaması, çalışma zamanında sınır denetimi, her alan için joker olmayan ölçütle test, fuzz testi ve kanaryayla başlayan **kademeli dağıtım**. Kademeli dağıtım bir test değildir; hatayı bulmaz, zararı sınırlar.

Güncel (Kasım 2025) — Cloudflare: 18 Kasım 2025'te bir veritabanı izin değişikliği, bot yönetiminin özellik dosyasını iki katına çıkardı; en çok 200 özellik bekleyen proxy kodu sınır aşılınca çöktü ve çekirdek trafik yaklaşık üç saat aksadı (Şekil 3). CrowdStrike'tan 16 ay sonra aynı zincir: kod değişmedi, değişen bir veriydi; tetikleyici koşul gizli hatayı arızaya çevirdi. Şirketin ilk önlemi de aynı: kendi ürettiği yapılandırma dosyalarını kullanıcı girdisi gibi sıkı denetlemek.



Şekil 3. Cloudflare'in olay raporu, 18 Kasım 2025 (blog.cloudflare.com; kısa alıntı, FSEK m. 35)

Okuma önerisi (açık erişimli): MIT 6.102 "Testing" okuması · *The Debugging Book*, "Introduction to Debugging" · *Software Engineering at Google*, Bölüm 11.

7. Okuma ve kaynaklar

Bu hafta okunacak: ISTQB CTFL v4.0.1 Türkçe müfredat, Bölüm 1 (s. 17–25). Turkish Testing Board'un sitesinden (turkishtestingboard.org) ücretsiz indirilir.

Gelecek hafta için: JDK 25 (Eclipse Temurin, adoptium.net; JDK 21 de olur) ve bir Java IDE'si (IntelliJ IDEA ya da VS Code) kurun. Ödev 1 (hata zinciri, test seviyeleri ve risk tabanlı test planı; bireysel, yalnız rapor) 2. haftada verilir, 3. haftanın Pazar günü 23.59'a kadar teslim edilir. Değerlendirme: vize %30, ödevler %20 (altı iki haftalık ödev %10 + dönem sonu ödevi %10), final %50; finalden en az 55 gerekir.

ISTQB (2024). *Certified Tester Foundation Level Syllabus v4.0.1* ve Türkçe çevirisi (Turkish Testing Board).

Turkish Testing Board (2026). *ISTQB Yazılım Testi Terimler Sözlüğü v2.0*.

CrowdStrike (2024). *External Technical Root Cause Analysis — Channel File 291*, 6 Ağustos 2024.

Prince, M. (2025). *Cloudflare outage on November 18, 2025*. Cloudflare Blog, 18 Kasım 2025. https://blog.cloudflare.com/18-november-2025-outage/ (erişim 9 Ekim 2026).

Lions, J. L. ve ark. (1996). *Ariane 5 Flight 501 Failure — Report by the Inquiry Board*, 19 Temmuz 1996.

Leveson, N. G. ve Turner, C. S. (1993). An investigation of the Therac-25 accidents. *IEEE Computer*, 26(7), 18–41.

U.S. Securities and Exchange Commission (2013). *In the Matter of Knight Capital Americas LLC*, Release No. 34-70694.

RTI (2002). *The Economic Impacts of Inadequate Infrastructure for Software Testing*. NIST Planning Report 02-3.

Krasner, H. (2022). *The Cost of Poor Software Quality in the US: A 2022 Report*. CISQ.

Stack Overflow (2025). *2025 Developer Survey*, AI bölümü.

PractiTest (2026). *The 2026 State of Testing Report*.

Boehm, B. ve Basili, V. R. (2001). Software defect reduction top 10 list. *IEEE Computer*, 34(1), 135–137.

Menzies, T., Nichols, W., Shull, F. ve Layman, L. (2017). Are delayed issues harder to resolve? *Empirical Software Engineering*, 22(4), 1903–1935.

Ammann, P. ve Offutt, J. (2016). *Introduction to Software Testing* (2. bs.). Cambridge University Press.

Beizer, B. (1990). *Software Testing Techniques* (2. bs.). Van Nostrand Reinhold.

MIT 6.102 Software Construction. *Testing* okuması. Massachusetts Institute of Technology, açık ders sayfaları.

Zeller, A. *The Debugging Book*. CISPA Helmholtz Center for Information Security. https://www.debuggingbook.org/ (CC BY-NC-SA 4.0).

Zeller, A., Gopinath, R., Böhme, M., Fraser, G. ve Holler, C. *The Fuzzing Book*. CISPA Helmholtz Center for Information Security. https://www.fuzzingbook.org/ (CC BY-NC-SA 4.0).

Bender, A. (2020). Testing Overview. T. Winters, T. Manshreck ve H. Wright (Ed.), *Software Engineering at Google*, Bölüm 11. O'Reilly. Açık sürüm (CC BY-NC-ND 4.0): https://abseil.io/resources/swe-book/html/ch11.html